

PeoplePay360

HR & Payroll — 16-Hour Odoo Hackathon Technical Plan, Architecture & Antigravity IDE Build Prompts

Prepared for: Team build session (MERN stack)

Scope: Problem statement + Excalidraw workflow analysis → prioritized MVP → build prompts

Format: Hackathon technical lead / product architect review

PeoplePay360: HR & Payroll — Hackathon Build Plan

1. Problem Understanding

A. The Real-World Problem

Most basic HR tools treat employee data, attendance, leave, and salary as separate, disconnected records. In reality, payroll is a **derived** process: it depends on which contract is active for an employee in a given period, what schedule that employee works, whether their attendance has exceptions, and what leave balances they've consumed. A tool that only offers CRUD screens for these entities without connecting them fails the moment payroll has to be run, because payroll math needs all of them to agree.

Who Experiences This Problem

- **HR Managers** — juggle employee lifecycle events (new hires, contract renewals, transfers) without a single source of truth.
- **Payroll Officers/Managers** — must manually reconcile attendance, leave, and contract data before running payroll, which is slow and error-prone.
- **Employees** — lack visibility into their own attendance, leave balance, and payslip history.
- **Finance/Admin leadership** — need aggregated, real-time visibility into salary cost, headcount, and payroll risk, not static spreadsheets.

Why the Problem Exists

- HR systems are often built module-by-module without shared foreign keys (Employee ↔ Contract ↔ Attendance ↔ Time Off ↔ Payslip).
- Payroll rules (structures/rules, sequencing, formulas) are often hardcoded instead of configurable, so every salary change needs a code change.
- Contract history isn't period-aware — systems let two contracts be "active" at once, which breaks payroll calculation for that period.
- Leave balances aren't automatically deducted from allocations when a request is approved, causing manual balance tracking.

Current Pain Points / Inefficiencies

- No unified employee "hub" that links to related records (contracts, attendance, leave).
- Payroll teams manually cross-check attendance and leave data before computing salary.
- Salary rules are static/hardcoded rather than configurable and sequenced.
- No visibility into payroll issues (missing bank info, duplicate payslips) before finalization.
- Dashboards, where they exist, use static/mock numbers instead of live aggregated data.

Expected Solution

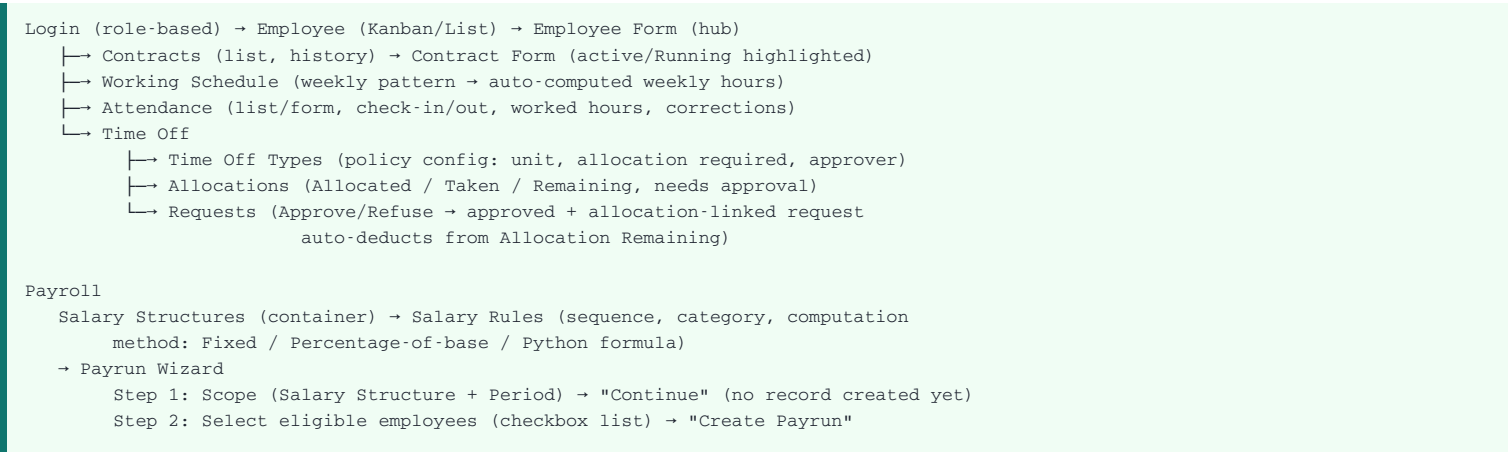
A connected, end-to-end HR & Payroll operational flow where the **Employee** record is the central hub; **Contracts** and **Working Schedules** supply payroll context; **Attendance** and **Time Off** capture daily HR activity; **Salary Structures/Rules** define computation; and a two-step **Payrun** wizard turns eligible employees into validated, downloadable, emailable **Payslips** — all visible through a live **Payroll Dashboard**.

Key Business Objectives

- Period-accurate contract selection for payroll (no concurrent "Running" contracts for the same employee/period).
- Configurable, sequenced Salary Rules that actually drive payslip line items (not static mockups).
- Two-step Payrun creation (scope → employee selection) that surfaces warnings before finalization.
- Automatic leave balance consumption when a Time Off request tied to an allocation is approved.
- A live dashboard aggregating Employee, Contract, Attendance, Time Off, and Payroll data.
- Role-based access across 5 distinct roles (Employee, HR Manager, HR Payroll User, HR Payroll Manager, Admin).

2. Workflow Analysis (Excalidraw Diagram → Textual Flow)

The diagram is a screen-by-screen functional walkthrough (not a strict swimlane diagram). It confirms and adds UI-level detail to the PDF problem statement. Converted into a single end-to-end textual flow:



(Payrun created ONLY with selected employees)

→ Payrun Processing Screen: Compute → Validate → Mark Paid → Send Payslips
(shows warnings: missing bank details, duplicate payslip, per-employee status)

→ Payslip Detail: rule-by-rule breakdown (Basic → Allowances → Gross → Deductions → Net) → Print Payslip (PDF)

Payroll Dashboard

Filters: Period, Department, Employee Type, Company

→ KPI cards (Net Salary Paid, Payslips Generated, Avg Salary, Approved Time Off, Attendance Health)

→ Charts (Salary Cost by Department, Monthly Net Salary Trend)

→ Alerts (missing bank info, duplicate payslips, unvalidated drafts, expiring contracts)

→ Attendance Overview, Time Off Overview, Department Overview

Step-by-Step Detail

Step	Actor	Input	Output	Decision / Status Point
View Employees	HR / Admin	Search/filter	Kanban or List of employees	—
Open Employee Form	HR / Admin / Employee(self)	Employee ID	Profile + smart buttons (counts) for Contracts/Attendance/Time Off	—
Manage Contracts	HR Manager	Employee, dates, wage, structure	Contract record (Draft/Running/Expired)	Only one Running contract per employee per period
Define Working Schedule	HR Manager/Admin	Day/Start/End/Break per weekday	Auto-computed weekly hours	—
Record Attendance	Employee (self) / System	Check-in/out timestamps	Worked hours, Status (Present/Late/Absent)	Manual correction restricted to HR/Admin
Configure Time Off Type	HR Manager/Admin	Unit, allocation required?, approver level	Policy definition	Determines whether a Request needs an Allocation
Allocate Leave	HR Manager	Employee, type, days granted	Allocation (needs its own approval)	Approved → balance becomes available
Submit Time Off Request	Employee	Type, dates, reason	Request (Draft/To Approve)	Approve / Refuse by HR Manager
Approve Request	HR Manager	Request	Status = Approved; linked Allocation "Remaining" reduced	Only applies if Type requires allocation
Configure Salary Structure/Rules	HR Payroll Manager	Rule name, code, category, sequence, computation method	Ordered rule set attached to a Structure	Sequence controls dependency order (e.g., Gross before Net)
Create Payrun — Step 1	HR Payroll User/Manager	Structure, Period	Wizard state (not persisted)	"Continue" ≠ record creation
Create Payrun — Step 2	HR Payroll User/Manager	Selected employees	Payrun record with linked draft Payslips	Only selected employees included
Compute Payrun	HR Payroll User/Manager	Payrun	Payslip line items per employee, using applicable contract	Warnings surfaced (missing bank info, duplicates)
Validate / Mark Paid	HR Payroll Manager	Computed Payrun	Status: Draft → Computed → Validated → Paid	Paid batches archived as history
Print / Send Payslip	HR Payroll User/Manager	Payslip / Payrun	PDF file; bulk email dispatch	—
View Dashboard	HR Payroll User/Manager, Admin	Filters (Period/Dept/Type)	KPI cards, charts, alerts	Must reflect live data, not static values

Diagram vs. Problem Statement — Conflicts & Resolution

Point	Diagram	Problem Statement	Resolution for Prototype
Currency in Payrun mock	Uses USD (\$) in the "Select Employee Records" popup	Uses ₹ (INR) throughout dashboard/payslip examples	Use ₹ consistently — the diagram's \$ example is incidental UI mockup styling, not a functional requirement.
User Management module	Shows a full Admin "User Management" screen (create user, assign roles, link to Employee)	Roles are described but a dedicated user-admin UI isn't explicitly required	Build a minimal version (seeded demo users with fixed roles) — full self-service user management is P2/P3, not core to judging.
Attendance quick-action widget	Shows a floating check-in/out widget with a live timer	Only requires attendance capture generally	Nice-to-have polish; implement a simplified check-in/out button, skip the live elapsed-timer animation if time is short.

No structural conflicts exist — the diagram is a faithful UI expansion of the written spec and should be treated as authoritative for screen layout and field-level detail.

3. Users & Roles

Role	Who	Needs	Key Actions	Sees	Decisions
Employee	Any staff member	Self-service visibility	View own profile, attendance, leave balance; create attendance/time-off requests; check in/out	Own records only	Whether to submit a leave request
HR Manager	HR staff	Manage HR master data	Full CRUD on Employees, Attendance, Contracts, Working Schedules, Time Off; approve/refuse leave	All employees' HR data (no payroll)	Approve/refuse Time Off; contract terms
HR Payroll User	Payroll executor	Run payroll operationally	All HR Manager rights + Create/Read/Update Payruns & Payslips; read-only Salary Structures/Rules	Payroll data, read-only salary config	Which employees to include in a Payrun; compute/validate
HR Payroll Manager	Payroll lead	Full payroll control	All HR Payroll User rights + full CRUD on Payruns, Payslips, Salary Structures, Salary Rules	Everything payroll-related	Salary rule design; final Mark Paid
Admin	System owner	Full system control	Full access to all modules; user management, role assignment	Everything	Who gets which role

4. Business Entities

Entity	Why It Matters	Key Relationships
User (Account)	Login identity, separate from Employee, carries Role(s)	1:1 with Employee (usually)
Employee	Central hub of the whole system	1:N Contracts, 1:N Attendance, 1:N Time Off Requests, 1:N Allocations, N:1 Department, N:1 Manager, N:1 Working Schedule
Department	Grouping for reporting/dashboard	1:N Employees
Contract	Period-specific employment terms; source of truth for payroll wage/structure	N:1 Employee, N:1 Salary Structure, N:1 Working Schedule
Working Schedule	Defines expected weekly hours; used by Attendance & Payroll	1:N Contracts/Employees
Attendance	Daily presence record; feeds dashboard and (optionally) payroll	N:1 Employee
Time Off Type	Policy definition (unit, allocation requirement, approver)	1:N Allocations, 1:N Requests
Allocation	Leave balance grant; must be approved before use	N:1 Employee, N:1 Time Off Type
Time Off Request	Leave transaction; approval consumes Allocation balance	N:1 Employee, N:1 Time Off Type, N:1 Allocation
Salary Structure	Named container of ordered Salary Rules	1:N Salary Rules, 1:N Contracts, 1:N Payruns
Salary Rule	Individual computation line (Basic, Allowance, Deduction, Gross, Net)	N:1 Salary Structure
Payrun	Batch representing one payroll period for selected employees	1:N Payslips, N:1 Salary Structure
Payslip	Computed salary document per employee per period	N:1 Payrun, N:1 Employee, N:1 Contract, contains N Payslip Lines
Payslip Line	One computed Salary Rule result on a Payslip	N:1 Payslip, N:1 Salary Rule
Notification (lightweight)	Alerts for approvals, payslip generation, warnings	N:1 User

5. Functional Modules

Modules are grouped by business function, not technical layer. Priority tags reflect Section 6.

M1 — Authentication & Role Management P0

Purpose	Role-based login and route protection for 5 roles
Users/Roles	All roles
Main Features	Login form, JWT session, protected routes, role-based menu visibility, seeded demo users (one per role)
Input	Email + password
Output	JWT token + role claim; redirect to role-appropriate dashboard
Workflow	Login → verify credentials → issue JWT with role → frontend stores token → route guard checks role per page
UI Screens	Login page; (P2) minimal User Management list for Admin
Required Data	User: email, passwordHash, role, employeeId
API	POST /api/auth/login, GET /api/auth/me

M2 — Employee & Department Management P0

Purpose	Central hub for employee master data
Users/Roles	HR Manager, HR Payroll User/Manager, Admin (full); Employee (self, read-only)
Main Features	List + Kanban views, Employee Form with smart-button counts linking to Contracts/Attendance/Time Off
Input	Name, email, department, manager, job position, working schedule, status
Output	Employee record; filtered related-record views
Workflow	Create/Edit Employee → assign Department/Manager/Schedule → navigate to related Contracts/Attendance/Time Off via smart buttons
UI Screens	Employee List, Employee Kanban, Employee Form
Required Data	Employee: name, email, phone, department, manager, jobPosition, workingSchedule, status, workLocation
API	GET/POST/PUT /api/employees, GET /api/employees/:id

M3 — Contract Management P0

Purpose	Track period-specific employment terms; single source of truth for payroll wage/structure
Users/Roles	HR Manager, HR Payroll (read for payroll use), Admin
Main Features	Contract history list highlighting active "Running" contract; wage, salary structure, working schedule assignment; business rule preventing two concurrent Running contracts per employee
Input	Employee, start/end date, wage, department, job position, salary structure
Output	Contract record with computed status (Draft/Running/Expired)
Workflow	Create Contract → validate no overlapping Running contract → status auto-set by date range
UI Screens	Contracts List (filtered per employee), Contract Form
Required Data	Contract: employeeId, startDate, endDate, wage, status, department, jobPosition, salaryStructureId, workingScheduleId
API	GET/POST/PUT /api/contracts, GET /api/employees/:id/contracts

M4 — Working Schedule P1

Purpose	Define weekly working pattern; auto-derive weekly hours
Users/Roles	HR Manager, Admin
Main Features	List + Form; per-day Start/End/Break; auto-computed total weekly hours
Input	Name, per-day start/end/break times
Output	Schedule with computed weekly hours, assignable to Employee/Contract
Workflow	Define days → system sums (end-start-break) per day → total weekly hours
UI Screens	Schedule List, Schedule Form
Required Data	WorkingSchedule: name, days[{day,start,end,break}], weeklyHours (computed), status
API	GET/POST/PUT /api/schedules

M5 — Attendance P0

Purpose	Capture daily presence; feed dashboard analytics
Users/Roles	Employee (create own), HR Manager/Admin (correct any record)

Main Features	Check-in/out, auto worked-hours calc, status (Present/Late/Absent), authorized manual correction
Input	Check-in / check-out timestamp
Output	Worked hours, overtime, status
Workflow	Employee checks in → checks out → system computes worked hours & status → HR can edit exceptions
UI Screens	Attendance List (global & per-employee), Attendance Form, Quick check-in/out widget (P2)
Required Data	Attendance: employeeId, checkIn, checkOut, workedHours, overtime, status, isManuallyEdited
API	GET/POST/PUT /api/attendance, POST /api/attendance/check-in, PUT /api/attendance/check-out

M6 — Time Off (Types, Allocations, Requests) P0

Purpose	Manage leave policy, balances, and approval lifecycle
Users/Roles	Employee (create request), HR Manager (approve/refuse, configure types & allocations), Admin
Main Features	Time Off Types (unit, allocation requirement, approver); Allocations (Allocated/Taken/Remaining); Requests (Approve/Refuse) with automatic balance deduction
Input	Type config; allocation grants; request dates/type/reason
Output	Approved/Refused request; updated Allocation.remaining
Workflow	Type defines rules → Allocation grants balance (needs approval) → Employee requests leave → HR approves/refuses → approval reduces linked Allocation.remaining
UI Screens	Time Off Types List/Form, Allocations List/Form, Requests List/Form
Required Data	TimeOffType: name, unit, requiresAllocation, approverLevel. Allocation: employeeId, typeId, allocated, taken, remaining, status. Request: employeeId, typeId, startDate, endDate, duration, status, allocationId
API	CRUD /api/timeoff-types, /api/allocations, /api/timeoff-requests, PUT /api/timeoff-requests/:id/approve, PUT /api/timeoff-requests/:id/refuse

M7 — Salary Structures & Salary Rules P0

Purpose	Configurable computation engine that actually drives payslips
Users/Roles	HR Payroll Manager (full CRUD), HR Payroll User (read-only), Admin
Main Features	Structures as ordered rule containers; Rules with category (Basic/Allowance/Gross/Deduction/Net), sequence, computation method (Fixed / Percentage-of-base / Formula)
Input	Rule name, code, category, sequence, computation type & value/expression
Output	Ordered rule list used by the Payslip computation engine
Workflow	Define Structure → attach Rules in sequence → Payrun selects Structure → engine evaluates rules in sequence order
UI Screens	Structures List/Form, Rules List/Form
Required Data	SalaryStructure: name, active. SalaryRule: structureId, name, code, category, sequence, computationType, value/percentageBase/formula
API	CRUD /api/salary-structures, /api/salary-rules

M8 — Payrun & Payslip Processing P0 (core demo)

Purpose	Turn eligible employees into computed, validated, paid payslips
Users/Roles	HR Payroll User (create/compute/read/update), HR Payroll Manager (full control incl. Mark Paid)
Main Features	Two-step wizard (scope → employee selection); Compute/Validate/Mark Paid/Send Payslips actions; warning surfacing; rule-by-rule payslip breakdown; PDF export; bulk email (simulated)
Input	Salary Structure, Period, selected employees
Output	Payrun with N Payslips, each with computed lines and Net Salary
Workflow	Step 1 scope → Step 2 select employees → Create Payrun (Draft) → Compute (runs rule engine per employee using their applicable Contract) → surfaces warnings → Validate → Mark Paid → Send Payslips
UI Screens	Payruns List, Payrun Wizard (2 steps), Payrun Processing screen, Payslips List, Payslip Detail
Required Data	Payrun: name, structureId, periodStart, periodEnd, status, employeeIds[]. Payslip: payrunId, employeeId, contractId, workedDays, lines[], gross, net, status, warnings[]
API	POST /api/payruns, GET /api/payruns, PUT /api/payruns/:id/compute, PUT /api/payruns/:id/validate, PUT /api/payruns/:id/mark-paid, POST /api/payruns/:id/send-payslips, GET /api/payslips/:id/pdf

M9 — Payroll Dashboard P0 (core demo)

Purpose	Live, filterable aggregation across all HR/Payroll data
Users/Roles	HR Payroll User/Manager, Admin (view); HR Manager (HR-only subset, P2)
Main Features	KPI cards, Salary Cost by Department chart, Monthly Net Salary Trend chart, Payroll Alerts, Attendance Overview, Time Off Overview, Department Overview — all filterable by Period/Department/Employee Type

Input	Filter selections
Output	Aggregated read-only metrics computed from live DB records
Workflow	Backend aggregation endpoint queries Employees/Contracts/Attendance/TimeOff/Payslips → returns computed JSON → frontend renders cards/charts
UI Screens	Payroll Dashboard (single scrollable page with filter bar)
Required Data	Derived only — no new collections; reads across Employee, Contract, Attendance, TimeOffRequest, Allocation, Payslip
API	GET /api/dashboard?period=&department=&employeeType=

6. Feature Prioritization (16-Hour Reality Check)

P0 — MUST HAVE

- Auth with 5 seeded demo users (one per role) and role-based route/menu guarding
- Employee List/Kanban + Employee Form with related-record smart buttons
- Contract CRUD with active-contract validation (one Running contract per employee per period)
- Attendance list + manual entry/correction (skip live check-in/out widget)
- Time Off Types, Allocations, Requests with Approve/Refuse and automatic balance deduction
- Salary Structures + Salary Rules (Fixed and Percentage-of-base computation types only)
- Payrun 2-step wizard → Compute → Validate → Mark Paid, with warning detection (missing bank info, duplicate payslip)
- Payslip detail screen with rule-by-rule breakdown
- Payslip PDF generation (single employee)
- Payroll Dashboard with live KPI cards + at least 2 charts, driven by real DB queries
- Realistic seed/demo data (employees, contracts, attendance, leave, one computed payrun)

P1 — SHOULD HAVE

- Working Schedule module (can hardcode "40 hrs/week" as default if time-constrained)
- Formula-based (Python-like expression) salary rule computation — implement a small safe-expression evaluator
- Send Payslips bulk-email action (can be simulated / logged rather than real SMTP)
- Attendance quick check-in/out widget with elapsed timer
- Full Attendance Overview + Time Off Overview + Department Overview blocks on dashboard

P2 — NICE TO HAVE

- Admin User Management screen (create user, assign role, link employee)
- Contract renewal reminders / "contracts expiring this month" alert
- Toast/notification center for approvals
- Kanban drag-and-drop status changes for Employees

P3 — DO NOT BUILD

- Real SMTP-based bulk payslip email delivery — simulate instead; not worth the setup time or judging value
- SSO / password reset / email verification flows
- Multi-company / multi-currency support
- Full audit-log / field-level history tracking
- Arbitrary Python code execution for salary rules (security risk + time sink) — use a constrained formula mini-language instead
- Mobile app / offline attendance sync

7. Final 16-Hour Prototype Scope

The prototype demonstrates one connected story: **an employee's daily HR activity (attendance, leave) combines with their contract to produce an accurate, validated payslip, visible in real time on a dashboard.** Everything else is supporting scaffolding.

8. Core Demo Flow

```
Login (as HR Payroll Manager)
  → Dashboard (show current live KPIs)
  → Employees → open "Aarav Mehta" → show Contracts / Attendance / Time Off smart buttons
  → Contracts → show Running contract with wage + structure
  → Time Off → Requests → approve a pending request → show Allocation "Remaining" drop
  → Payroll → Salary Structures → show "Regular Salary" with sequenced rules
  → Payruns → New → Step 1 (structure + period) → Continue → Step 2 (select employees)
    → Create Payrun
  → Payrun screen → Compute → show warnings (missing bank info) → Validate → Mark Paid
  → Open a Payslip → show rule-by-rule breakdown → Print Payslip (PDF downloads)
  → Back to Dashboard → filters change → Net Salary Paid / charts update live
```

Screens required: Login, Dashboard, Employee List/Form, Contract List/Form, Attendance List, Time Off Requests/Allocations/Types, Salary Structures/Rules, Payrun Wizard + Processing, Payslip Detail.

APIs required: auth, employees, contracts, attendance, timeoff-types/allocations/requests, salary-structures/rules, payruns (+compute/validate/mark-paid), payslips (+pdf), dashboard.

DB collections: users, employees, departments, contracts, workingSchedules, attendance, timeOffTypes, allocations, timeOffRequests, salaryStructures, salaryRules, payruns, payslips.

Status transitions: Contract: Draft→Running→Expired. TimeOffRequest: Draft→ToApprove→Approved/Refused. Payrun/Payslip: Draft→Computed→Validated→Paid.

Demo/mock data required: ~10-15 employees across 3-4 departments, 2 contracts per employee (1 expired, 1 running), 2 weeks of attendance, a mix of leave requests/allocations, one fully processed Payrun with an intentional warning (missing bank detail) to show off warning detection.

Real vs. Demo/Mock Functionality

Capability	Treatment
Contract → payroll period selection	REAL — actual date-range logic
Salary rule sequencing & computation (fixed + percentage)	REAL — actual engine
Leave balance deduction on approval	REAL
Payroll warning detection (missing bank info, duplicates)	REAL — simple rule checks
Dashboard aggregation	REAL — live MongoDB aggregation queries
Payslip PDF generation	REAL
Bulk email "Send Payslips"	DEMO — button triggers a success toast + status change; no real SMTP
Formula-based salary rules (advanced)	DEMO/SIMPLIFIED — small safe-expression evaluator, not full Python
Attendance live check-in widget with timer	DEMO — simplified button, skip live ticking timer if short on time
User Management (Admin)	DEMO — seeded users only, no self-service UI unless time remains

9. Application Architecture (MERN)

Frontend (React)

Pages: Login, Dashboard, Employees (List/Kanban/Form), Contracts (List/Form), Attendance (List/Form), TimeOff (Types/Allocations/Requests), Payroll (Structures/Rules/Payruns/Payslips)

Components: Sidebar Nav, Topbar, DataTable, StatusBadge, KpiCard, ChartCard, Modal/Wizard, Form fields, ConfirmDialog, Toast

State: React Context (auth/user) + useState/useReducer per page;
React Query (or simple fetch+useEffect) for server data caching

API layer: /src/api/*.js — one file per resource, axios instance with JWT interceptor

Backend (Node.js + Express)

Routes → /routes/*.routes.js (one per resource)

Controllers → /controllers/*.controller.js (request handling, validation)

Services → /services/payrollEngine.service.js (rule computation),
/services/dashboard.service.js (aggregation) — only where logic is non-trivial; simple CRUD skips the service layer

Models → /models/*.model.js (Mongoose schemas)

Middleware → auth.middleware.js (JWT verify), role.middleware.js (RBAC)

Database

MongoDB via Mongoose. See Section 10 for full schema.

10. Database Design (MongoDB / Mongoose)

Collection	Key Fields	Relationships / Notes
users	email, passwordHash, role (enum), employeeId (ref)	1:1 Employee
departments	name	1:N employees
employees	name, email, phone, departmentId, managerId (ref employees), jobPosition, workingScheduleId, status, workLocation	self-ref manager; ref department, workingSchedule
workingSchedules	name, days:[{day,start,end,break}], weeklyHours, status	referenced by employees/contracts
contracts	employeeId, startDate, endDate, wage, status(Draft/Running/Expired), departmentId, jobPosition, salaryStructureId, workingScheduleId	index on (employeeId, status) to enforce single Running contract
attendance	employeeId, date, checkIn, checkOut, workedHours, overtime, status(Present/Late/Absent), isManuallyEdited	index on (employeeId, date)
timeOffTypes	name, unit(Days/Hours), requiresAllocation(bool), approverLevel	referenced by allocations/requests
allocations	employeeId, typeId, allocated, taken, remaining, status, validity	N:1 employee, N:1 type
timeOffRequests	employeeId, typeId, startDate, endDate, duration, status(Draft/ToApprove/Approved/Refused), allocationId, reason, approverId	on Approve: decrement linked allocation.remaining
salaryStructures	name, active	1:N salaryRules
salaryRules	structureId, name, code, category(Basic/Allowance/Gross/Deduction/Net), sequence, computationType(Fixed/Percentage/Formula), value, percentageBase, formula	ordered by sequence within structure
payruns	name, salaryStructureId, periodStart, periodEnd, status(Draft/Computed/Validated/Paid), employeeIds:[ref]	1:N payslips
payslips	payrunId, employeeId, contractId, workedDays, lines:[{ruleId,name,code,category,amount}], gross, net, status, warnings:[string]	N:1 payrun, N:1 employee, N:1 contract

11. API Design (Key REST Endpoints)

POST /api/auth/login

GET /api/auth/me

GET /api/employees

POST /api/employees

GET /api/employees/:id/contracts

GET /api/contracts

PUT /api/contracts/:id

GET /api/schedules

POST /api/schedules

GET /api/employees/:id

PUT /api/employees/:id

POST /api/contracts

GET	/api/attendance	POST /api/attendance
PUT	/api/attendance/:id	
GET	/api/timeoff-types	POST /api/timeoff-types
GET	/api/allocations	POST /api/allocations
GET	/api/timeoff-requests	POST /api/timeoff-requests
PUT	/api/timeoff-requests/:id/approve	
PUT	/api/timeoff-requests/:id/refuse	
GET	/api/salary-structures	POST /api/salary-structures
GET	/api/salary-rules	POST /api/salary-rules
POST	/api/payruns	(Step 2 – creates record)
GET	/api/payruns	GET /api/payruns/:id
PUT	/api/payruns/:id/compute	
PUT	/api/payruns/:id/validate	
PUT	/api/payruns/:id/mark-paid	
POST	/api/payruns/:id/send-payslips	
GET	/api/payslips	GET /api/payslips/:id
GET	/api/payslips/:id/pdf	
GET	/api/dashboard?period=&department=&employeeType=	

12. UI Screens

Screen	Role	Key Elements	Priority
Login	All	Email/password form, error state	P0
Dashboard	Payroll roles, Admin	Filter bar, 5 KPI cards, 2 charts, alerts list, tables	P0
Employee List / Kanban	HR, Payroll, Admin	Search, status filter, avatar cards, NEW button	P0
Employee Form	HR, Payroll, Admin, self	Profile fields, smart-button counts, tabs (Work/Private)	P0
Contracts List / Form	HR, Admin	Status badges (Running highlighted), wage, dates	P0
Working Schedule List / Form	HR, Admin	Weekly grid, auto-computed hours	P1
Attendance List / Form	HR, Admin, self	Check-in/out columns, status badges, correction form	P0
Time Off Types / Allocations / Requests	HR, Admin, self (requests)	Approve/Refuse buttons, balance columns	P0
Salary Structures / Rules	Payroll Manager (edit), Payroll User (view)	Sequence table, computation type selector	P0
Payrun Wizard (2-step)	Payroll User/Manager	Step indicator, structure+period form, employee checklist	P0
Payrun Processing	Payroll User/Manager	Compute/Validate/Mark Paid/Send buttons, payslip summary table, warning banners	P0
Payslip Detail	Payroll roles, self (own)	Rule-by-rule table, Print/PDF button	P0
User Management	Admin	User list, role assignment form	P2

Common patterns across all screens: empty states ("No records yet — click NEW to create one"), error states (inline banner on failed fetch/save), loading skeletons, confirmation dialogs for destructive actions (e.g., deleting a contract), and consistent status badge coloring (green=Active/Paid, amber=Draft/Pending, red=Refused/Warning, gray=Expired/Inactive).

13. Antigravity IDE Prompts

Ten copy-paste-ready prompts, ordered by build dependency. Each is self-contained context for the Antigravity IDE agent.

PROMPT 1 — Project Foundation

CONTEXT

We are building "PeoplePay360" — an integrated HR & Payroll MERN prototype for a 16-hour hackathon. This is the first prompt: it sets up the whole project skeleton that every later module will build on.

OBJECTIVE

Initialize a MERN monorepo with a clean, professional SaaS/ERP-style shell (sidebar nav, topbar, routing, theme) but no business logic yet.

USER ROLES

Employee, HR Manager, HR Payroll User, HR Payroll Manager, Admin (roles are just enum values for now — real auth comes in Prompt 2).

FUNCTIONAL REQUIREMENTS

- /client (React + Vite) and /server (Node + Express) folders.
- React Router with placeholder pages: Dashboard, Employees, Contracts, Attendance, TimeOff, Payroll (Structures, Rules, Payruns, Payslips).
- Persistent Sidebar with icons + labels for each top-level menu (Employees ▼, Contracts, Attendance, Time Off ▼, Payroll ▼), collapsible sub-menus for Time Off (Requests/Allocations/Types) and Payroll (Payruns/Payslips/Structures/Rules).
- Topbar with app name, current user avatar placeholder, logout button (no-op for now).
- Global theme: professional SaaS palette (deep navy/teal primary, neutral grays, one accent color), consistent spacing scale, card + table base styles.
- Express server with a health-check route (GET /api/health) and CORS enabled for the client origin.
- Mongoose connection setup using an environment variable MONGO_URI.

UI REQUIREMENTS

Clean sidebar navigation, top navbar, a reusable PageHeader component (title + primary action button), a reusable DataTable shell component, a reusable KpiCard shell component. No mock business data yet — just the shell rendering correctly on every route.

BACKEND REQUIREMENTS

Express app entry (server.js), central error-handling middleware, a /config/db.js for the Mongoose connection, a /routes/health.routes.js.

DATABASE REQUIREMENTS

None yet beyond the connection itself.

BUSINESS LOGIC

None yet.

DEMO DATA

None yet.

EDGE CASES

Handle Mongo connection failure gracefully with a clear console error, don't crash silently.

DESIGN REQUIREMENTS

Modern SaaS/ERP look — think a polished internal ops tool, not a generic AI dashboard. Clear typographic hierarchy, generous whitespace, rounded cards with subtle shadows, consistent 8px spacing grid.

IMPLEMENTATION RULES

Inspect the existing project before making changes. Keep the folder structure modular (client/src/{pages,components,api,context}, server/{routes,controllers,models,middleware,services}). Do not add business logic yet — this prompt is scaffolding only. Avoid unnecessary dependencies.

PROMPT 2 — Authentication & Role-Based Access

Prerequisite: Prompt 1 (Project Foundation) complete.

CONTEXT

PeoplePay360 needs 5 distinct roles with different access levels: Employee, HR Manager, HR Payroll User, HR Payroll Manager, Admin. This prompt adds real authentication and route protection on top of the Prompt 1 shell.

OBJECTIVE

Implement JWT-based login, a User model linked to Employee, role-based route guarding on both frontend and backend, and 5 seeded demo users (one per role) so judges can switch roles instantly during the demo.

USER ROLES

All 5 roles, as defined in the problem statement's Section 3 (User Roles).

FUNCTIONAL REQUIREMENTS

- Backend: User model {email, passwordHash, role, employeeId}. POST /api/auth/login validates credentials and returns a JWT containing {userId, role, employeeId}. GET /api/auth/me returns the current user from the token.
- Middleware: auth.middleware.js verifies JWT; role.middleware.js(...allowedRoles) restricts a route to specific roles.
- Frontend: Login page posts to /api/auth/login, stores JWT in memory + a React Context (AuthContext), attaches it via an axios interceptor to every request.
- ProtectedRoute component redirects to /login if unauthenticated, and hides/disables sidebar items the current role cannot access (e.g., Employee role never sees Payroll menu; HR Manager never sees Payruns/Structures/Rules).
- After login, redirect every role to the Dashboard (Employee role sees a simplified personal view — full dashboard logic comes later in Prompt 3).

UI REQUIREMENTS

A clean centered Login card ("HR Portal — Welcome back") with email/password fields and an inline error message on failed login. Sidebar menu items conditionally rendered based on role.

BACKEND REQUIREMENTS

POST /api/auth/login, GET /api/auth/me, JWT signing with an env secret, bcrypt password hashing.

DATABASE REQUIREMENTS

users collection: email (unique), passwordHash, role (enum: employee, hr_manager, hr_payroll_user, hr_payroll_manager, admin), employeeId (ref, nullable for now).

BUSINESS LOGIC

- Only Admin can (eventually) create/edit other users — for this prototype, just seed 5 fixed demo accounts via a seed script; a full user-management UI is out of scope (P2).
- Users must never be able to elevate their own role — since there's no self-service role editor yet, this is naturally satisfied.

DEMO DATA

Seed exactly 5 users, one per role, e.g.: employee@peoplepay360.com, hrmanager@peoplepay360.com, payrolluser@peoplepay360.com, payrollmanager@peoplepay360.com, admin@peoplepay360.com — all with password "Demo@123" for judging convenience.

EDGE CASES

Invalid credentials show a clear inline error, not a console-only failure. Expired/missing token redirects cleanly to /login instead of crashing the app.

DESIGN REQUIREMENTS

Login screen should look like a polished enterprise portal, matching the shell's theme from Prompt 1.

IMPLEMENTATION RULES

Reuse the shell/theme from Prompt 1. Do not rewrite the routing structure — extend it with guards. Keep JWT secret in an environment variable, never hardcoded.

PROMPT 3 — Employee, Department & Contract Module (Core Business Module, Part 1)

Prerequisite: Prompt 2 (Auth & Roles) complete.

CONTEXT

The Employee record is the central hub of PeoplePay360 – every other module (Contracts, Attendance, Time Off, Payroll) links back to it. This prompt builds Employees, Departments, and Contracts together since Contracts cannot exist without Employees.

OBJECTIVE

Implement full CRUD for Employees and Contracts, with the Employee Form acting as a navigation hub via smart-button counts to related records.

USER ROLES

HR Manager, HR Payroll User/Manager, Admin: full CRUD on Employees & Contracts.
Employee: read-only access to their own record only.

FUNCTIONAL REQUIREMENTS

- Employee List (table) and Kanban view, both opening the same Employee Form.
- Employee Form: identity fields (name, email, phone), Work Information tab (department, manager, job position, working schedule reference – schedule itself can be a simple dropdown of hardcoded values until Prompt for Working Schedule lands, if built), status; Private Information tab (basic placeholder fields). Smart buttons at the top show live counts: "Contracts (n)", "Attendance (n)", "Time Off (n)" – clicking opens that related list, pre-filtered to this employee.
- Contracts List: shows Contract number, Employee, Start, End, Wage/Month, Status – the "Running" status contract is visually highlighted (e.g., green badge + bold row).
- Contract Form: employee, start date, end date, wage, department, job position, salary structure (dropdown – can be a stubbed list until Prompt 5 lands), status (computed, not manually set).
- Business rule: when saving a contract with status "Running", the backend must reject the save if the same employee already has another "Running" contract whose date range overlaps the new one – return a clear validation error.

UI REQUIREMENTS

List + Kanban toggle for Employees. Filtered "related record" list views reachable only via smart buttons (no separate top-level page needed for "employee's contracts", it's the same Contracts list pre-filtered).

BACKEND REQUIREMENTS

GET/POST/PUT /api/employees, GET /api/employees/:id,
GET /api/employees/:id/contracts, GET/POST/PUT /api/contracts.
Server-side overlap validation on contract save.

DATABASE REQUIREMENTS

employees: name, email, phone, departmentId, managerId (ref employees),
jobPosition, workingScheduleId (nullable), status, workLocation.
departments: name (seed 4-5: Finance, HR, Engineering, Sales, Support).
contracts: employeeId, startDate, endDate, wage, status(Draft/Running/Expired),
departmentId, jobPosition, salaryStructureId (nullable for now), workingScheduleId (nullable).

BUSINESS LOGIC

Contract status: Draft (future start date), Running (current date within range, no end date or end date in future), Expired (end date in past). One Running contract per employee per overlapping period – enforce server-side.

DEMO DATA

Seed 10-15 employees across the 4-5 departments with realistic Indian names, each with 1-2 contracts (most employees: 1 expired + 1 running, showing history).

EDGE CASES

Saving a contract that would create two concurrent Running contracts for the same employee must fail with a clear error message shown in the form.

DESIGN REQUIREMENTS

Employee cards show initials avatar, name, job title, department, status dot.
Contract list clearly distinguishes Running (green) vs Expired (gray) vs Draft (amber) statuses.

IMPLEMENTATION RULES

Reuse the DataTable and PageHeader components from Prompt 1. Keep Employee and Contract as separate models/routes/controllers even though built in one prompt.
Do not hardcode employee data in the frontend – always fetch from the API.

PROMPT 4 — Attendance & Working Schedule

Prerequisite: Prompt 3 (Employee & Contract) complete.

CONTEXT

Attendance and Working Schedule capture the day-to-day operational side of HR that later feeds both the Payroll Dashboard and (optionally) payroll calculation.

OBJECTIVE

Build Attendance tracking (list, form, corrections) and a simplified Working Schedule module.

USER ROLES

Employee: create/view own attendance. HR Manager/Admin: view all, correct any record. HR Payroll roles: read-only for reporting purposes.

FUNCTIONAL REQUIREMENTS

- Working Schedule List/Form: name, per-weekday start/end/break time, with auto-computed total weekly hours (sum of (end-start-break) per active day). Assignable to an Employee or Contract via a dropdown.
- Attendance List: global view and a per-employee filtered view (reached via the Employee Form's smart button). Columns: Employee, Check In, Check Out, Worked Hours, Status.
- Attendance Form: shows one record's detail; Worked Hours and Overtime are computed automatically from Check In/Check Out vs. the employee's assigned schedule; manual correction of Check In/Check Out is allowed only for HR Manager/Admin roles.
- Simple "Check In" / "Check Out" action buttons for the logged-in Employee role (P1 – build a simplified static button rather than a live ticking timer widget if time is tight).

UI REQUIREMENTS

Attendance list with clear status badges (Present=green, Late=amber, Absent=red). Working Schedule form shows a weekly grid identical in spirit to a timetable (Day / Start / End / Break / Hours columns).

BACKEND REQUIREMENTS

GET/POST/PUT /api/schedules, GET/POST/PUT /api/attendance,
POST /api/attendance/check-in, PUT /api/attendance/:id/check-out.

DATABASE REQUIREMENTS

workingSchedules: name, days:[{day, start, end, break}], weeklyHours (computed on save), status.
attendance: employeeId, date, checkIn, checkOut, workedHours, overtime,
status(Present/Late/Absent), isManuallyEdited(bool).

BUSINESS LOGIC

Status derivation: no check-in by a cutoff time → Absent; check-in after a threshold → Late; otherwise Present. Worked hours = checkOut - checkIn (minus schedule break if applicable). Overtime = worked hours beyond scheduled hours for that day.

DEMO DATA

Seed 2 weeks of attendance for all seeded employees, including a few Late/Absent records and 1-2 manually-corrected records, so the dashboard has realistic variety.

EDGE CASES

Missing check-out (still "open" attendance) should display worked hours as 0.00 or "-" until closed, not crash the computation.

DESIGN REQUIREMENTS

Consistent status badge system shared with other modules (reuse the StatusBadge component).

IMPLEMENTATION RULES

Reuse existing DataTable/Form components. Do not duplicate the status-badge logic – extend the shared component from Prompt 1/3.

PROMPT 5 — Time Off (Types, Allocations, Requests) — Workflow / Approval System

Prerequisite: Prompt 3 (Employee) complete; independent of Prompt 4.

CONTEXT

Time Off is one of the two demo-critical workflows (the other is Payroll). It must show a believable approval lifecycle with real balance math.

OBJECTIVE

Implement Time Off Types (policy config), Allocations (balance grants), and Requests (leave transactions with Approve/Refuse), with approved allocation-linked requests automatically reducing the balance.

USER ROLES

Employee: create own requests, view own allocations/balance. HR Manager: full CRUD on Types/Allocations, Approve/Refuse Requests. HR Payroll roles: read-only (for dashboard purposes). Admin: full access.

FUNCTIONAL REQUIREMENTS

- Time Off Types List/Form: name, unit (Days/Hours), requiresAllocation (bool), approverLevel (Manager/Officer), active flag.
- Allocations List/Form: employee, type, allocated, taken (computed), remaining (computed), status (needs its own approval before the balance becomes usable).
- Time Off Requests List/Form: employee, type, start/end date, computed duration, status (Draft/To Approve/Approved/Refused), reason, linked allocation (if the type requires one). Approve/Refuse buttons visible to HR Manager/Admin only.
- Business rule: approving a request whose type requiresAllocation must find the employee's matching approved Allocation and decrement its "remaining" by the request's duration; if remaining would go negative, block approval with a clear warning.
- All three (Types/Allocations/Requests) are reached only via a "Time Off ▼" navbar dropdown — no separate top-level pages.

UI REQUIREMENTS

Requests list shows clear status pills. Allocation list exposes Allocated/Taken/Remaining at a glance. Request Form shows Approve/Refuse buttons only to authorized roles, disabled otherwise.

BACKEND REQUIREMENTS

CRUD for /api/timeoff-types, /api/allocations, /api/timeoff-requests, plus PUT /api/timeoff-requests/:id/approve and PUT /api/timeoff-requests/:id/refuse.

DATABASE REQUIREMENTS

timeOffTypes: name, unit, requiresAllocation, approverLevel, active.
allocations: employeeId, typeId, allocated, taken, remaining, status, validity.
timeOffRequests: employeeId, typeId, startDate, endDate, duration, status, allocationId (nullable), reason, approverId.

BUSINESS LOGIC

On Approve (with allocation): remaining -= duration, taken += duration. On Refuse: no balance change, status = Refused. Duration is computed from start/end date based on the type's unit.

DEMO DATA

Seed 3 Time Off Types (Paid Time Off — Days/requires allocation/Manager approval; Sick Leave — Days/no allocation/Manager; Comp Off — Hours/requires allocation/Officer). Seed allocations and a mix of Approved/To Approve requests across employees, matching the numbers shown in the workflow diagram for realism.

EDGE CASES

Approving a request for a type that requires allocation but has no matching Allocation record should show a clear error rather than silently succeeding.

DESIGN REQUIREMENTS

Consistent status-pill styling with Contracts/Attendance modules.

IMPLEMENTATION RULES

Group these three sub-modules under one "Time Off" feature folder on both frontend and backend for maintainability, but keep them as separate Mongoose models/collections.

PROMPT 6 — Salary Structures & Salary Rules (Payroll Configuration Engine)

Prerequisite: Prompt 3 (Contracts, for salaryStructureId reference) complete.

CONTEXT

This is the configuration layer that makes Payroll real instead of hardcoded. Salary Structures group ordered Salary Rules; a Payrun's selected Structure determines exactly which rules run for every payslip in that batch.

OBJECTIVE

Build CRUD for Salary Structures and Salary Rules, plus a small, safe computation engine (a service function, not yet wired to Payruns — that happens in Prompt 7) that can evaluate a structure's rules in sequence for one employee/contract.

USER ROLES

HR Payroll Manager: full CRUD. HR Payroll User: read-only. Others: no access.

FUNCTIONAL REQUIREMENTS

- Salary Structures List/Form: name, active flag, and a rules sub-table showing attached rules in sequence order with drag-or-numeric reordering.
- Salary Rules List/Form: name, code, category (Basic/Allowance/Gross/Deduction/Net), sequence, computationType (Fixed Amount / Percentage of Base / Formula), and the relevant value field: fixedAmount, OR percentageValue + percentageBase (which prior category/rule to base the percentage on, e.g. "Basic"), OR a constrained formula string using only previously computed rule codes as variables (e.g. "BASIC + HRA - PF"), evaluated with a safe expression parser — never raw eval() or Python code execution.
- payrollEngine.service.js: a pure function computeSalary(structure, contract) that iterates rules by sequence, resolves each computation type, and returns an ordered list of {code, name, category, amount} plus computed gross and net.

UI REQUIREMENTS

Structure Form shows its rules in a clean sequence table. Rule Form's fields change dynamically based on the selected computation type (show/hide relevant inputs).

BACKEND REQUIREMENTS

CRUD /api/salary-structures, /api/salary-rules. A standalone, unit-testable payrollEngine.service.js used later by the Payrun module.

DATABASE REQUIREMENTS

salaryStructures: name, active.

salaryRules: structureId, name, code (unique per structure), category, sequence, computationType, fixedAmount, percentageValue, percentageBaseCode, formula.

BUSINESS LOGIC

Rules execute strictly in ascending sequence order. Percentage-of-base rules reference an earlier rule's computed amount by its code — if that code hasn't been computed yet (wrong sequencing), throw a clear configuration error rather than silently returning 0. Formula rules use a restricted expression evaluator (e.g. mathjs or a hand-rolled safe parser) limited to +, -, *, /, and references to already-computed rule codes — no arbitrary code execution.

DEMO DATA

Seed a "Regular Salary" structure matching the workflow diagram: Basic Salary (Percentage of Wage, 50%), House Rent Allowance (Percentage of Basic, 40%), Standard Allowance (Fixed), Gross Salary (Formula: BASIC+HRA+STD), Provident Fund (Percentage of Basic, deduction), Professional Tax (Fixed, deduction), Net Salary (Formula: GROSS-PF-PT).

EDGE CASES

Two rules with the same sequence number in one structure should be rejected on save with a validation message.

DESIGN REQUIREMENTS

Sequence numbers and categories should be visually scannable (small colored category chips: Basic=blue, Allowance=green, Deduction=red, Gross/Net=navy).

IMPLEMENTATION RULES

Keep the computation engine as a pure, side-effect-free function so it can be reused directly by the Payrun compute action in Prompt 7 without modification. Never use eval() or exec() for formula evaluation.

PROMPT 7 — Payrun & Payslip Processing (Core Business Workflow)

Prerequisite: Prompts 3, 6 (Employees/Contracts, Salary Engine) complete.

CONTEXT

This is the single most important module for judging — it ties Employees, Contracts, and Salary Structures together into a real payroll run.

OBJECTIVE

Implement the two-step Payrun creation wizard, the Payrun processing screen (Compute/Validate/Mark Paid/Send Payslips), and the Payslip detail screen with PDF export.

USER ROLES

HR Payroll User: create/compute/validate. HR Payroll Manager: all of the above plus Mark Paid. Others: no access (Employee role may eventually view only their own payslip — Pl, add if time allows).

FUNCTIONAL REQUIREMENTS

- Payrun Wizard Step 1: choose Salary Structure and Period (start/end date). Clicking "Continue" only stores this in local wizard state — it must NOT create a database record yet.
- Payrun Wizard Step 2: shows eligible employees (those with a Running contract overlapping the chosen period) as a checkbox list with Working Hours/Start Date/Wage columns for context. "Create Payrun" is the only action that persists a Payrun record, and it must include only the checked employees.
- Payrun Processing screen: shows Payrun name/structure/period/status and a table of its Payslips (Employee, Warning, Worked, Basic, Gross, Net, Status). Action buttons: Compute, Validate, Mark Paid, Send Payslips — each only enabled when the Payrun is in the correct prior status (Draft→Compute→Validate→Mark Paid).
- Compute action: for each employee in the Payrun, find their applicable (Running, period-overlapping) Contract, run payrollEngine.computeSalary() from Prompt 6 using the Payrun's Salary Structure and that Contract's wage, and save the resulting Payslip with computed lines, gross, net.
- Warning detection during Compute: flag "A/C missing" if the employee has no bank details recorded (simulate with an isBankVerified boolean on Employee for simplicity), and "Duplicate" if another Payslip already exists for the same employee + overlapping period.
- Payslip Detail screen: Employee, Structure, Pay Run, Period, Status, Worked Days, and a full rule-by-rule Salary Computation table (Rule, Category, Amount, Code), ending in Net Salary. Includes a "Print Payslip" button that generates a PDF.
- Send Payslips (on the parent Payrun): simulated bulk action — mark each Payslip as "sent" and show a success toast; no real email delivery required.

UI REQUIREMENTS

Step indicator on the wizard (Step 1 of 2). Warning banners in red/amber on the Payrun processing screen. Payslip table with clear color-coded Status column.

BACKEND REQUIREMENTS

POST /api/payruns (Step 2 submission — creates the record),
GET /api/payruns, GET /api/payruns/:id,
PUT /api/payruns/:id/compute, PUT /api/payruns/:id/validate,
PUT /api/payruns/:id/mark-paid, POST /api/payruns/:id/send-payslips,
GET /api/payslips, GET /api/payslips/:id, GET /api/payslips/:id/pdf.

DATABASE REQUIREMENTS

payruns: name, salaryStructureId, periodStart, periodEnd, status(Draft/Computed/Validated/Paid), employeeIds:[ref].
payslips: payrunId, employeeId, contractId, workedDays, lines:[{ruleCode, ruleName, category, amount}], gross, net, status, warnings:[string], sentAt.

BUSINESS LOGIC

Status flow strictly: Draft → Computed (after Compute) → Validated (after Validate) → Paid (after Mark Paid). Each transition should be blocked from the backend if attempted out of order, with a clear error. Paid Payruns become read-only (no further edits) and remain visible as history.

DEMO DATA

Seed at least one fully processed Payrun (status=Paid) with 4-5 Payslips matching the workflow diagram's numbers (Aarav Mehta ₹75k net, Sara Khan with an "A/C missing" warning, John Dsouza with a "Duplicate" warning left in Draft), plus one Payrun left in Draft status to show the wizard/compute flow live during the demo.

EDGE CASES

Attempting to Mark Paid a Payrun that hasn't been Validated should be blocked with a clear message. An employee with no applicable Running contract for the chosen period should be excluded from eligible-employee selection in Step 2, with a tooltip explaining why.

DESIGN REQUIREMENTS

This is the visual centerpiece of the demo — invest extra polish here: clear step indicator, satisfying status-change animations/toasts, a professional payslip layout that looks print-ready.

IMPLEMENTATION RULES

Reuse payrollEngine.service.js from Prompt 6 without modification — this module should only orchestrate calling it, not reimplement salary math. Use a PDF library (e.g., pdfkit or puppeteer-based HTML-to-PDF) for Print Payslip.

PROMPT 8 — Payroll Dashboard (Reporting / Analytics)

Prerequisite: Prompts 3, 4, 5, 7 (all data-producing modules) complete.

CONTEXT

The dashboard is the second demo centerpiece — it must visibly aggregate REAL data from every other module, not display static numbers.

OBJECTIVE

Build a single Payroll Dashboard page with filters (Period, Department, Employee Type) and live-computed KPI cards, charts, alerts, and overview tables.

USER ROLES

HR Payroll User/Manager, Admin (full dashboard). HR Manager (P1: HR-only subset without salary figures, if time allows).

FUNCTIONAL REQUIREMENTS

- Filter bar: Period (month/year picker), Department (dropdown incl. "All"), Employee Type (dropdown incl. "All"), Company (static, single value for this prototype).
- KPI cards: Total Net Salary Paid (+ % vs previous period), Payslips Generated (paid vs pending breakdown), Average Salary per Employee, Approved Time Off Days, Attendance Health (%).
- Charts: Salary Cost by Department (bar chart), Monthly Net Salary Trend (line chart, last 6 periods).
- Payroll Alerts panel: computed from live data — employees missing bank details, duplicate payslip warnings, unvalidated drafts, contracts expiring within 30 days.
- Attendance Overview: Present/Late/Absent/Overtime counts, missing check-outs, manual edits count, attendance coverage %.
- Time Off Overview: per-type Approved Days / Pending / Remaining Balance table.
- Department Overview: Headcount + Monthly Salary per department table.
- ALL numbers must come from a single backend aggregation endpoint querying the actual collections — no hardcoded/mock numbers in the frontend.

UI REQUIREMENTS

KPI cards in a responsive grid at the top, charts below, alerts/overview tables in a lower grid. Match the workflow diagram's layout intent closely.

BACKEND REQUIREMENTS

GET /api/dashboard?period=&department=&employeeType= — a single endpoint that runs the necessary Mongo aggregation pipelines across employees, contracts, attendance, timeOffRequests, allocations, and payslips, and returns one consolidated JSON payload shaped for direct chart/card consumption.

DATABASE REQUIREMENTS

No new collections — pure read-side aggregation across existing ones.

BUSINESS LOGIC

Attendance Health % = present-or-corrected records / total expected records for the period. Salary Cost by Department = sum of Payslip.net grouped by the employee's department for payslips within the selected period. Trend chart uses the last 6 periods' totals regardless of current filter (trend is always company-wide over time).

DEMO DATA

Relies entirely on data seeded in Prompts 3/4/5/7 — no additional seeding needed if those prompts seeded realistically, but verify totals look non-trivial (double up seed volume if the dashboard looks sparse).

EDGE CASES

If no Payslips exist yet for the selected period, KPI cards should show "No data for this period" rather than \$0/blank/crash.

DESIGN REQUIREMENTS

This is a judging showpiece — invest in clean chart styling (use a chart library consistent with the frontend-design guidance), consistent color coding for alerts (red=critical, amber=warning, blue=info).

IMPLEMENTATION RULES

Compute everything server-side in the aggregation endpoint; the frontend should only be rendering, not calculating totals — this avoids inconsistency between screens.

PROMPT 9 — Demo Data / Seeding

Prerequisite: All data-model prompts (3, 4, 5, 6, 7) complete.

CONTEXT

Judges will spend most of their attention on a populated, realistic system. This prompt consolidates and finalizes all seed data into one reliable script.

OBJECTIVE

Create a single idempotent seed script (npm run seed) that wipes and repopulates all collections with a coherent, realistic dataset spanning every module, so the demo never starts from an empty state.

USER ROLES

N/A (build-time tooling).

FUNCTIONAL REQUIREMENTS

- One script (server/scripts/seed.js) that seeds, in dependency order: users (5 roles) → departments → workingSchedules → employees (12-15, varied departments/managers) → contracts (1-2 per employee, realistic history) → attendance (2 weeks per employee, with some Late/Absent/manually-edited records) → timeOffTypes (3) → allocations (per employee/type) → timeOffRequests (mix of Approved/To Approve/Refused) → salaryStructures + salaryRules (at least "Regular Salary" fully configured) → payruns + payslips (at least one Paid batch with a warning example, one Draft batch for live demo).
- All names, emails, and figures should look like real Indian company data (matching the tone of the workflow diagram: Aarav Mehta, Sara Khan, John Dsouza, Neha Patel, etc.) — extend to 12-15 total employees for a fuller dashboard.
- Script must be safely re-runnable (clears existing seeded data first).

UI REQUIREMENTS

None (backend-only).

BACKEND REQUIREMENTS

A single seed.js invoked via `node server/scripts/seed.js` or an npm script, using the same Mongoose models as the app (no duplicate schema definitions).

DATABASE REQUIREMENTS

Populates every collection defined in Prompts 2-7.

BUSINESS LOGIC

Must respect all validation rules already built (e.g., no two concurrent Running contracts for one employee) so the seed itself proves the business rules work.

DEMO DATA

This prompt IS the demo data — see Functional Requirements above for the exact shape.

EDGE CASES

Running the seed script twice in a row must not create duplicates or crash on unique-index violations.

DESIGN REQUIREMENTS

N/A.

IMPLEMENTATION RULES

Do not hardcode demo data directly into frontend components — everything must come from this seed script into the real database, so the dashboard and all modules demonstrate genuinely live computation, not display tricks.

PROMPT 10 — Final Integration & Polish

Prerequisite: All previous prompts (1-9) complete.

CONTEXT

This is the final pass before the judging demo — connecting any loose ends and making the whole app feel like one coherent product.

OBJECTIVE

Ensure frontend and backend are fully wired end-to-end, add loading/error/empty states everywhere, fix visual inconsistencies, and rehearse the core demo flow without errors.

USER ROLES

All roles — verify the experience for each seeded demo user.

FUNCTIONAL REQUIREMENTS

- Walk the full Core Demo Flow (Login → Dashboard → Employee → Contracts → Time Off approval → Salary Structure → Payrun wizard → Compute/Validate/Mark Paid → Payslip PDF → back to Dashboard showing updated numbers) end-to-end and fix any broken links or failed API calls found along the way.
- Add consistent loading skeletons/spinners to every data-fetching screen.
- Add consistent empty states ("No employees yet — click NEW to add one") to every list view.
- Add consistent error banners for failed API calls (never a silent blank screen).
- Add confirmation dialogs before destructive actions (deleting a contract, refusing a request, marking a Payrun as Paid).
- Verify role-based menu/route restrictions actually hide/block the right things for each of the 5 seeded users.
- Responsive check: sidebar collapses gracefully on narrower viewports used for judging screens/projectors.
- Fix any duplicate or unused components discovered along the way; consolidate shared UI elements.

UI REQUIREMENTS

Visual QA pass across every screen for consistent spacing, typography, color usage, and button styles — remove anything that looks like a "generic AI dashboard" (mismatched fonts, inconsistent card shadows, misaligned tables).

BACKEND REQUIREMENTS

Verify every route has proper error handling and returns sensible HTTP status codes (400 for validation errors, 401/403 for auth issues, 404 for missing records, 500 only for genuine server errors).

DATABASE REQUIREMENTS

None new — verify indexes exist where used for filtering (employeeId+date on attendance, employeeId+status on contracts).

BUSINESS LOGIC

Re-verify the two hardest business rules end-to-end: (1) no concurrent Running contracts, (2) approved allocation-linked leave correctly reduces balance — these are the rules judges are most likely to probe.

DEMO DATA

Confirm the seed script (Prompt 9) still produces a clean, populated demo state after all Prompt 10 fixes.

EDGE CASES

Test what happens on a completely fresh database before seeding (should show clean empty states, not crashes) and immediately after seeding (should show full data).

DESIGN REQUIREMENTS

Final professional polish pass — this is what judges see first and last.

IMPLEMENTATION RULES

Do not introduce new features in this prompt — only fix, connect, and polish what Prompts 1-9 built. Keep changes modular and easy to roll back if something breaks close to the demo deadline.

14. Prompt Execution Order

#	Prompt	Prerequisite
1	Project Foundation	None
2	Authentication & Roles	Prompt 1
3	Employee, Department & Contract	Prompt 2
4	Attendance & Working Schedule	Prompt 3
5	Time Off (Types/Allocations/Requests)	Prompt 3
6	Salary Structures & Rules	Prompt 3
7	Payrun & Payslip Processing	Prompts 3 & 6
8	Payroll Dashboard	Prompts 3, 4, 5, 7
9	Demo Data / Seeding	Prompts 3, 4, 5, 6, 7
10	Final Integration & Polish	Prompts 1-9

Prompts 4, 5, and 6 are mutually independent once Prompt 3 is done — they can be built in parallel by different team members.

15. 16-Hour Development Plan

Time Block	Duration	Focus
Hr 0 - 1	1h	Planning: confirm scope, assign Prompts 4/5/6 to parallel workstreams
Hr 1 - 2	1h	Prompt 1 — Project Foundation
Hr 2 - 3	1h	Prompt 2 — Auth & Roles
Hr 3 - 5.5	2.5h	Prompt 3 — Employee & Contract (core, do not rush)
Hr 5.5 - 7.5	2h (parallel)	Prompt 4 (Attendance) + Prompt 5 (Time Off) by two team members
Hr 5.5 - 7.5	2h (parallel)	Prompt 6 (Salary Structures/Rules engine) by a third member
Hr 7.5 - 10.5	3h	Prompt 7 — Payrun & Payslip Processing (core demo workflow, largest time block)
Hr 10.5 - 12.5	2h	Prompt 8 — Payroll Dashboard
Hr 12.5 - 13.5	1h	Prompt 9 — Demo Data / Seeding
Hr 13.5 - 15.5	2h	Prompt 10 — Integration, bug fixing, visual polish
Hr 15.5 - 16	0.5h	Demo rehearsal, backup plan (screenshots/recording) in case of live-demo failure

The majority of time (3h) is deliberately allocated to Payrun/Payslip processing, since it is the single workflow judges will scrutinize most closely, followed by the Dashboard and the Employee/Contract foundation.

16. Features to Cut First if Running Late

1. Formula-based salary rules — fall back to Fixed + Percentage only (still demonstrates a real configurable engine).
2. Attendance quick check-in/out widget with live timer — replace with a simple "Mark Present" button.
3. Time Off Overview / Department Overview dashboard blocks — keep only the 5 KPI cards + 1 chart if severely time-constrained.
4. Working Schedule module — hardcode "40 hrs/week" for all employees.
5. Send Payslips bulk email — reduce to a toast + status flag with zero backend email logic.
6. Admin User Management UI — rely entirely on the 5 seeded demo accounts.

What absolutely must be completed before the final demo: Login with role switching; Employee → Contract linkage; at least one Time Off approval reducing a balance; Salary Structure driving at least Fixed + Percentage rules; a full Payrun Compute → Validate → Mark Paid cycle with one visible warning; a Payslip PDF; and a Dashboard showing at least 3 live KPI numbers that visibly change when filters change.

17. Final Demo Strategy

- **Open on the Dashboard** logged in as HR Payroll Manager — it's the most visually impressive screen and immediately shows the system is "alive" with real numbers.
- **Tell the story, not the feature list:** "Here's an employee, here's their contract, here's a leave they took that reduced their balance, and here's exactly how that turns into their payslip." Judges remember narratives, not menus.
- **Demonstrate the two-step Payrun wizard live** rather than only showing an already-Paid one — this proves the workflow isn't faked, and is the moment to call out the warning detection (missing bank info / duplicate payslip).
- **Open the Payslip and narrate the rule sequence** (Basic → HRA → Gross → PF/PT → Net) to prove the Salary Rule engine is real, not hardcoded numbers.
- **Close back on the Dashboard**, changing the Period/Department filter live to show the numbers actually update — this is the

strongest signal of "connected system" vs. "disconnected screens."

- **Have a fallback:** a short screen recording of the full flow in case of live network/demo issues, plus the seed script pre-run so the database is never empty at judging time.
- **Mention the roadmap briefly** (per the problem statement's "Future roadmap" deliverable): formula-based salary rules, real email delivery, self-service user management, and mobile attendance — signals awareness of what was intentionally scoped out, not missed.